

Lecture 3

Logistic Regression

Outline

- **Logistic Regression**
- **Gradient descent for Logistic Regression**
- **Newton's Method for Logistic Regression**
- **Multinomial Logistic Regression**

Binary Classification

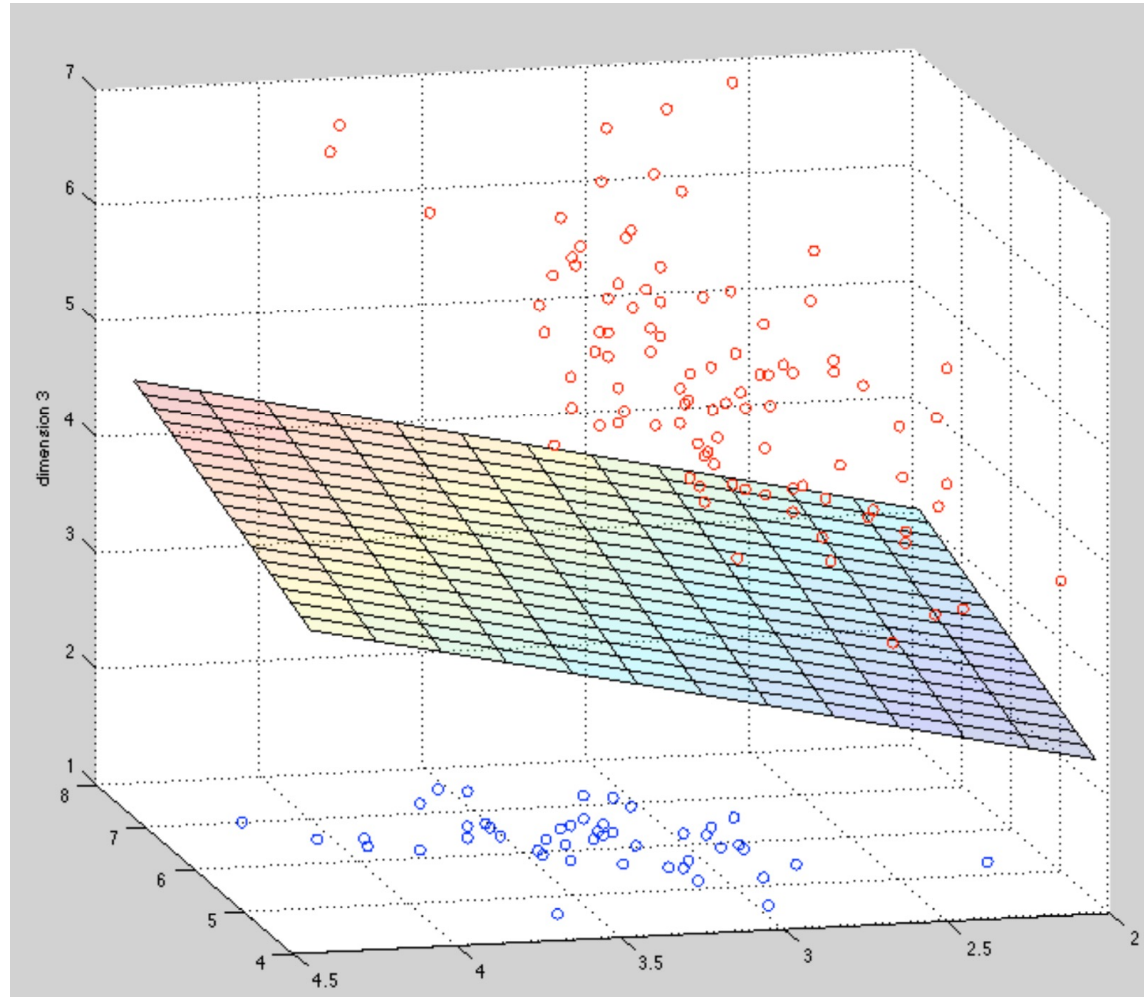
Suppose we're distinguishing cat from dog images



Two Phrases of Logistic Regression

- **Training:** we learn weights w and b using **stochastic gradient descent** and **cross-entropy loss**.
- **Test:** Given a test example x we compute $p(y|x)$ using learned weights w and b , and return whichever label ($y = 1$ or $y = 0$) is higher probability

Hyperplanes



Cross-Entropy in Deep Learning

python

复制

```
import torch
import torch.nn as nn

# 定义模型 (假设是一个简单的线性分类器)
class SimpleClassifier(nn.Module):
    def __init__(self, input_size, num_classes):
        super().__init__()
        self.fc = nn.Linear(input_size, num_classes)

    def forward(self, x):
        return self.fc(x)

# 初始化模型和损失函数
model = SimpleClassifier(input_size=10, num_classes=3)
criterion = nn.CrossEntropyLoss()

# 示例输入
batch_size = 4
inputs = torch.randn(batch_size, 10) # 输入特征 (batch_size, input_size)
labels = torch.tensor([0, 2, 1, 0]) # 真实标签 (batch_size,)

# 前向传播
outputs = model(inputs) # 输出 logits (batch_size, num_classes)

# 计算损失
loss = criterion(outputs, labels)
print(f"Loss: {loss.item():.4f}")
```



Cross-Entropy in LLM

LLM Next Token Prediction Loss Function

python

复制

```
import torch.nn as nn

# Model outputs logits of shape (batch_size, seq_len, vocab_size)
logits = model(input_ids)

# Targets are shifted input ids (shape: batch size, seq len)
loss_fn = nn.CrossEntropyLoss(ignore_index=pad_token_id)
loss = loss_fn(logits[:, :-1, :].reshape(-1, vocab_size), input_ids[:, 1:].reshape(-1))
```

$$\mathcal{L} = \frac{1}{N} \sum_{t=1}^n \text{CE}(p_t, y_t), \quad \text{CE}(p_t, y_t) = - \sum_{i=1}^V y_{t,i} \log(p_{t,i}),$$

Key idea behind today's lecture

1. Define a linear classifier (**logistic regression**)
2. Define an objective function (**likelihood**)
3. Optimize it with **gradient descent** to learn parameters
4. Predict the class with highest probability under the model

Binary Classification

- The predictions and the output labels

$$z = \mathbf{w}^T \mathbf{x} + b \quad y \in \{0, 1\} \quad \text{Bernoulli Distribution}$$

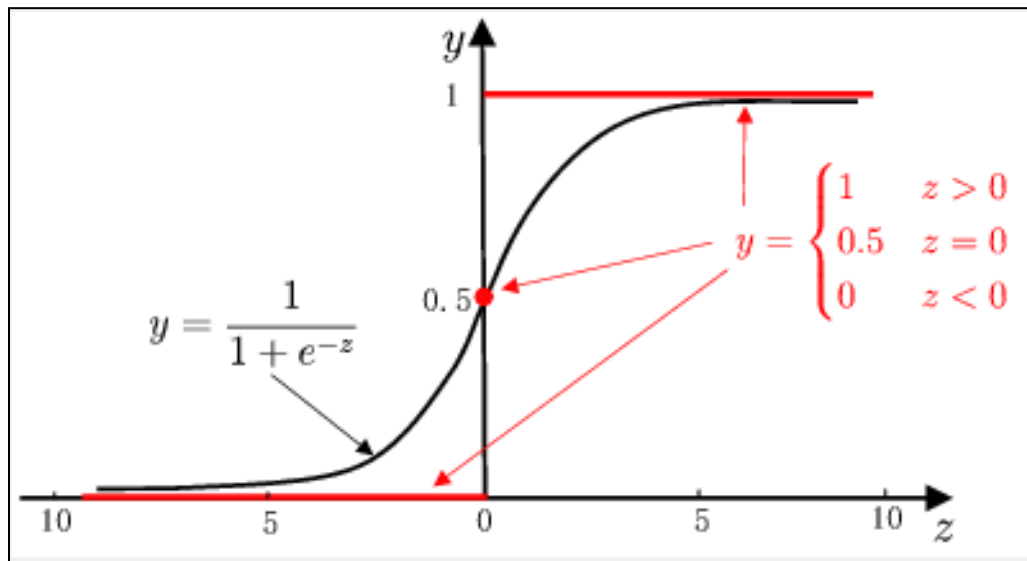
- The real-valued predictions of the linear regression model need to be converted into **0/1**.
- Ideally, the unit-step function is desired

$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$

- which predicts positive for z greater than 0, negative for z smaller than 0, and an arbitrary output when z equals to 0.

Binary Classification

- Disadvantages of unit-step function
 - not continuous
- Logistic (sigmoid) function: a surrogate function to approximate the unit-step function
 - monotonic differentiable



Comparison between unit-step function and logistic function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

Logistic Regression

Data: Inputs are continuous vectors of length d . Outputs are discrete labels.

$$\mathcal{D} = \left\{ \mathbf{x}^{(i)}, y^{(i)} \right\}_{i=1}^m \text{ where } \mathbf{x} \in \mathbb{R}^d \text{ and } y \in \{0, 1\}$$

Model: Logistic function applied to dot product of parameters with input vector.

$$p_{\theta}(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^T \mathbf{x})}$$

Learning: finds the parameters that minimize some objective function.

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Prediction: Output is the most probable class.

$$\hat{y} = \operatorname{argmax}_{y \in \{0, 1\}} p_{\theta}(y \mid \mathbf{x})$$

Logistic regression - maximum likelihood

*In statistics, **maximum likelihood estimation (MLE)** is a method of estimating the parameters of a statistical model given observations, by finding the parameter values that maximize the likelihood of making the observations given the parameters.*

MLE can be seen as a special case of the maximum a posteriori estimation (MAP) that assumes a uniform prior distribution of the parameters, or as a variant of the MAP that ignores the prior and which therefore is unregularized.

Logistic regression - maximum likelihood

- Maximum likelihood

- Given the training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$
- Maximizing the probability of each sample being predicted as the ground-truth label
 - the log-likelihood to be maximized is:

$$\ell(\mathbf{w}, b) = \log \prod_{i=1}^m p(y_i \mid \mathbf{x}_i; \mathbf{w}, b)$$

- assumption that the training examples are independent:

$$\ell(\mathbf{w}, b) = \sum_{i=1}^m \log p(y_i \mid \mathbf{x}_i; \mathbf{w}, b)$$

Log odds

- Apply logistic function

$$y = \frac{1}{1 + e^{-z}} \quad \text{transform into} \quad y = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

- Log odds

- the logarithm of the relative likelihood of a sample being a positive sample

$$\ln \frac{y}{1 - y} = \mathbf{w}^T \mathbf{x} + b$$

- Logistic regression has several nice properties

- without requiring any prior assumptions on the data distribution
- it predicts labels together with associated probabilities
- it is solvable with numerical optimization methods.

Logistic regression - maximum likelihood

- Log odds can be rewritten as

$$\ln \frac{p(y = 1 \mid \mathbf{x})}{p(y = 0 \mid \mathbf{x})} = \mathbf{w}^T \mathbf{x} + b$$

and consequently,

$$p(y = 1 \mid \mathbf{x}) = \frac{e^{\mathbf{w}^T \mathbf{x} + b}}{1 + e^{\mathbf{w}^T \mathbf{x} + b}} = \text{sigmoid}(\mathbf{w}^T \mathbf{x} + b)$$

$$\begin{aligned} p(y = 0 \mid \mathbf{x}) &= \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x} + b}} = 1 - \text{sigmoid}(\mathbf{w}^T \mathbf{x} + b) \\ &= \text{sigmoid}(-(\mathbf{w}^T \mathbf{x} + b)) \end{aligned}$$

Logistic regression - maximum likelihood

- Transform into minimize negative log-likelihood

- Let $\beta = (w; b)$, $\hat{x} = (x; 1)$, $w^T x + b$ can be rewritten as $\beta^T \hat{x}$

- Let $p_1(\hat{x}_i; \beta) = p(y = 1 \mid \hat{x}; \beta)$

$$p_0(\hat{x}_i; \beta) = p(y = 0 \mid \hat{x}; \beta) = 1 - p_1(\hat{x}_i; \beta)$$

the likelihood term in can be rewritten as

$$p(y_i \mid x_i; w, b) = y_i p_1(\hat{x}_i; \beta) + (1 - y_i) p_0(\hat{x}_i; \beta)$$

- maximizing log-likelihood is equivalent to minimizing

$$J(\beta) = \sum_{i=1}^m \left(-y_i \beta^T \hat{x}_i + \log \left(1 + e^{\beta^T \hat{x}_i} \right) \right)$$

Logistic regression - maximum likelihood

- Transform into minimize negative log-likelihood

- Let $\beta = (w; b)$, $\hat{x} = (x; 1)$, $w^T x + b$ can be rewritten as $\beta^T \hat{x}$

- Let $p_1(\hat{x}_i; \beta) = p(y = 1 \mid \hat{x}; \beta)$

$$p_0(\hat{x}_i; \beta) = p(y = 0 \mid \hat{x}; \beta) = 1 - p_1(\hat{x}_i; \beta)$$

the likelihood term in can be rewritten as

$$p(y_i \mid \hat{x}_i; w, b) = p_1(\hat{x}_i; \beta)^{y_i} p_0(\hat{x}_i; \beta)^{1-y_i}$$

- maximizing log-likelihood is equivalent to minimizing

$$J(\beta) = \sum_{i=1}^m -[y_i \log p_1(\hat{x}_i; \beta) + (1 - y_i) \log p_0(\hat{x}_i; \beta)]$$

The Cross-Entropy loss!

Maximum Likelihood Estimation

Learning: Four approaches to solving $\beta^* = \arg \min_{\beta} J(\beta)$

- ❑ **Approach 1:** Closed Form???
(set derivatives equal to zero and solve for parameters)
- ❑ **Approach 2:** Gradient Descent
(take larger – more certain – steps opposite the gradient)
- ❑ **Approach 3:** Stochastic Gradient Descent (SGD)
(take many small steps opposite the gradient)
- ❑ **Approach 4:** Newton's Method
(use second derivatives to better follow curvature)

Maximum Likelihood Estimation

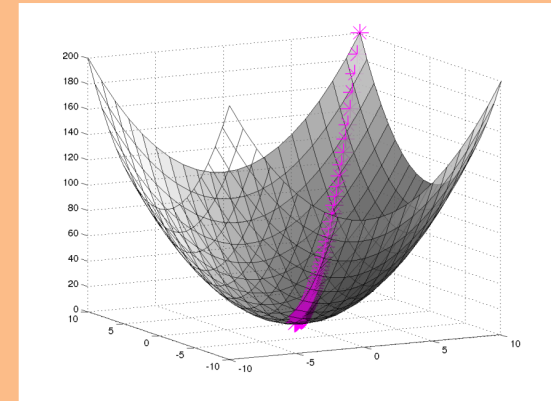
Learning: Four approaches to solving $\beta^* = \arg \min_{\beta} J(\beta)$

- ~~❑ **Approach 1:** Closed Form???~~
(set derivatives equal to zero and solve for parameters)
- ❑ **Approach 2:** Gradient Descent
(take larger – more certain – steps opposite the gradient)
- ❑ **Approach 3:** Stochastic Gradient Descent (SGD)
(take many small steps opposite the gradient)
- ❑ **Approach 4:** Newton's Method
(use second derivatives to better follow curvature)

Gradient Descent

Algorithm 1 Gradient Descent

```
1: procedure GD( $\mathcal{D}$ ,  $\theta^{(0)}$ )  
2:    $\theta \leftarrow \theta^{(0)}$   
3:   while not converged do  
4:      $\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta)$   
5:   return  $\theta$ 
```



$$\nabla_{\theta} J(\theta) = \begin{bmatrix} \frac{d}{d\theta_1} J(\theta) \\ \frac{d}{d\theta_2} J(\theta) \\ \vdots \\ \frac{d}{d\theta_N} J(\theta) \end{bmatrix}$$

$$\theta^{t+1} = \theta^t - \eta \nabla J_{\theta}(\theta)$$

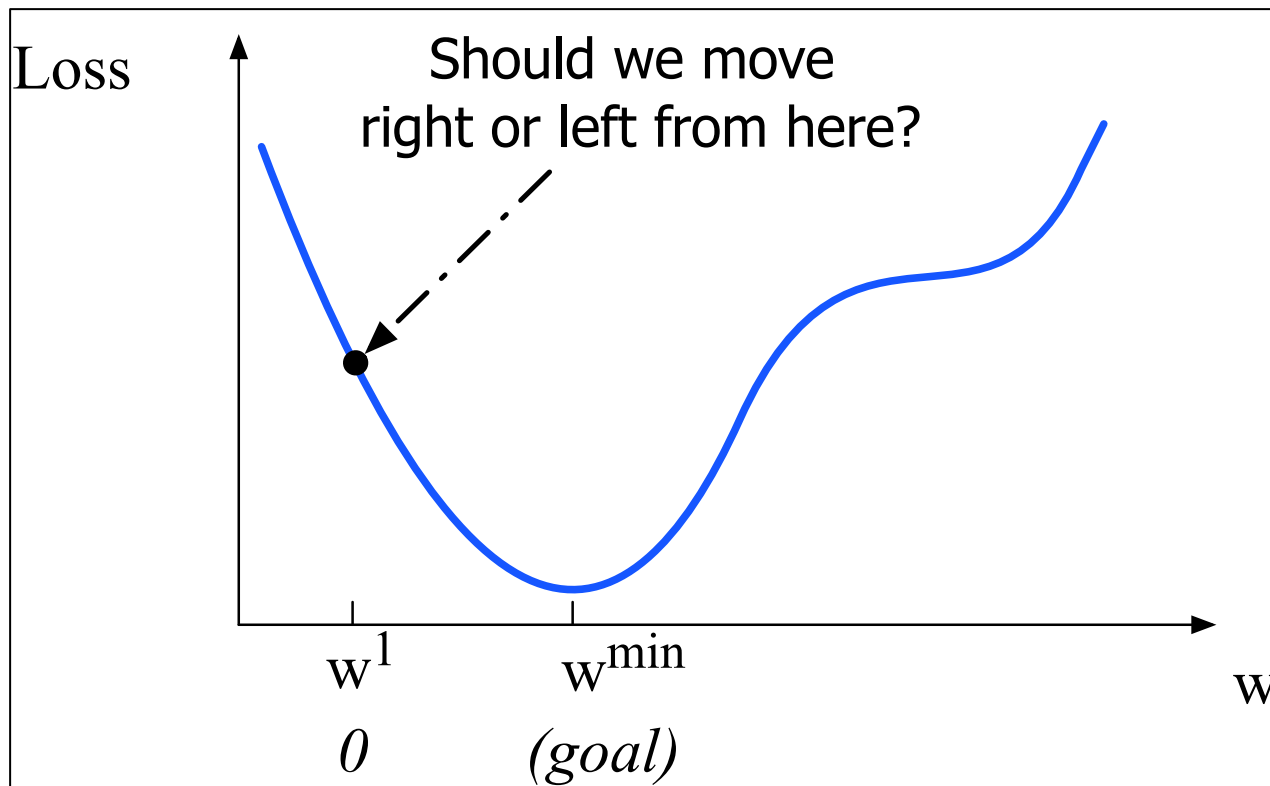
Review: Derivative of a Function

$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$ is called the derivative of f at x .

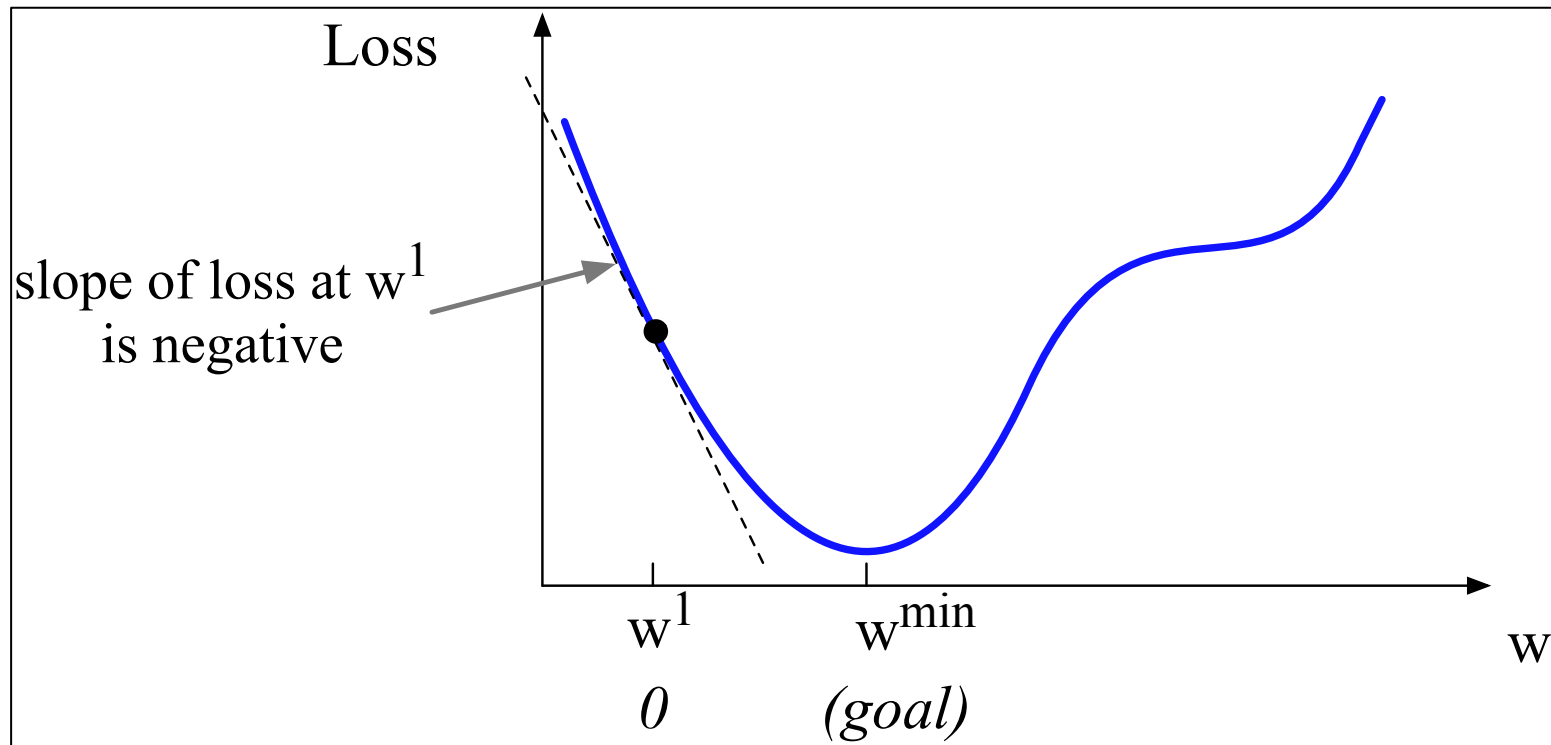
We write: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

“The derivative of f with respect to x is ...”

Gradient Descent



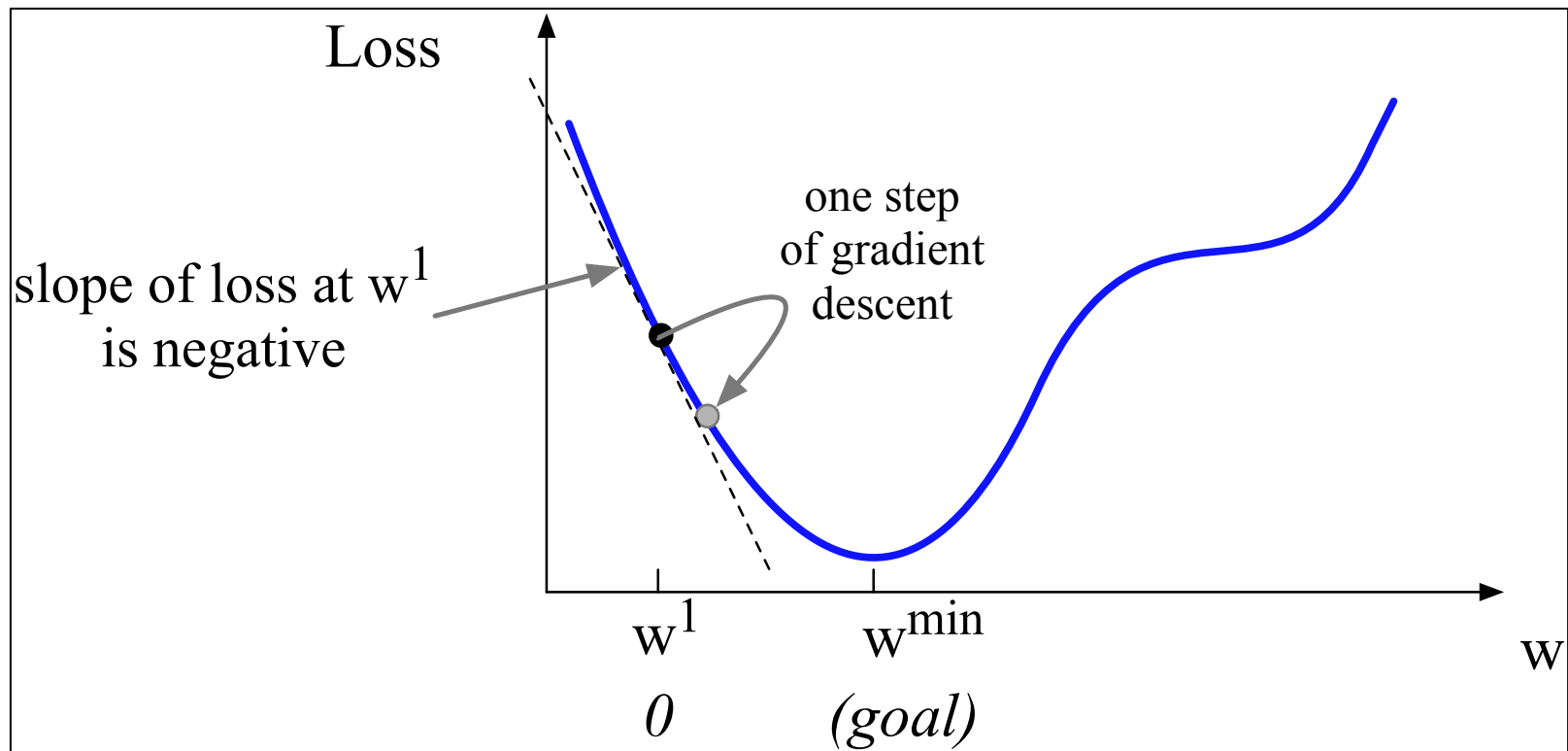
Gradient Descent



Gradient Descent

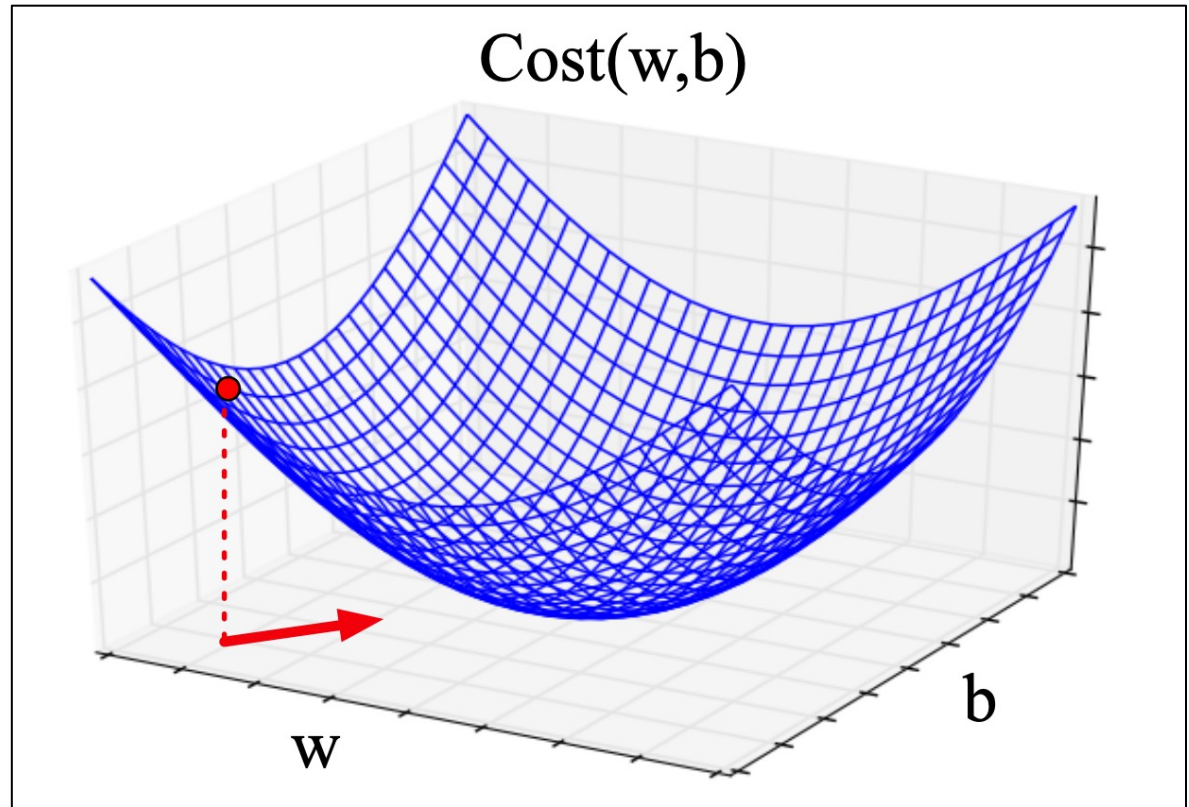
Q: Given current w , should we make it bigger or smaller?

A: Move w in the reverse direction from the slope of the function



Gradient Descent

- Visualizing the gradient vector at the red point
- It has two dimensions shown in the x-y plane



Online Resource

- Machine Learning Lecture 12 "Gradient Descent / Newton's Method"
- <https://www.youtube.com/watch?v=o6FfdP2uYh4>
- Instructor: Kilian Weinberger @ Cornell

Gradient for Logistic Regression

- The **cross-entropy** loss function

$$J(\boldsymbol{\beta}) = \sum_{i=1}^m -[y_i \log p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) + (1 - y_i) \log p_0(\hat{\mathbf{x}}_i; \boldsymbol{\beta})]$$

- The gradient

$$\frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = - \sum_{i=1}^m \hat{\mathbf{x}}_i (y_i - p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}))$$

- Instead of using the sum notation, we can more efficiently compute the gradient in its matrix form

$$\frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = \mathbf{X}(\sigma(\mathbf{X}^T \boldsymbol{\beta}) - \mathbf{y})$$

$\mathbf{X} \in \mathbb{R}^{d \times m}$
 $\sigma : \text{sigmoid}$

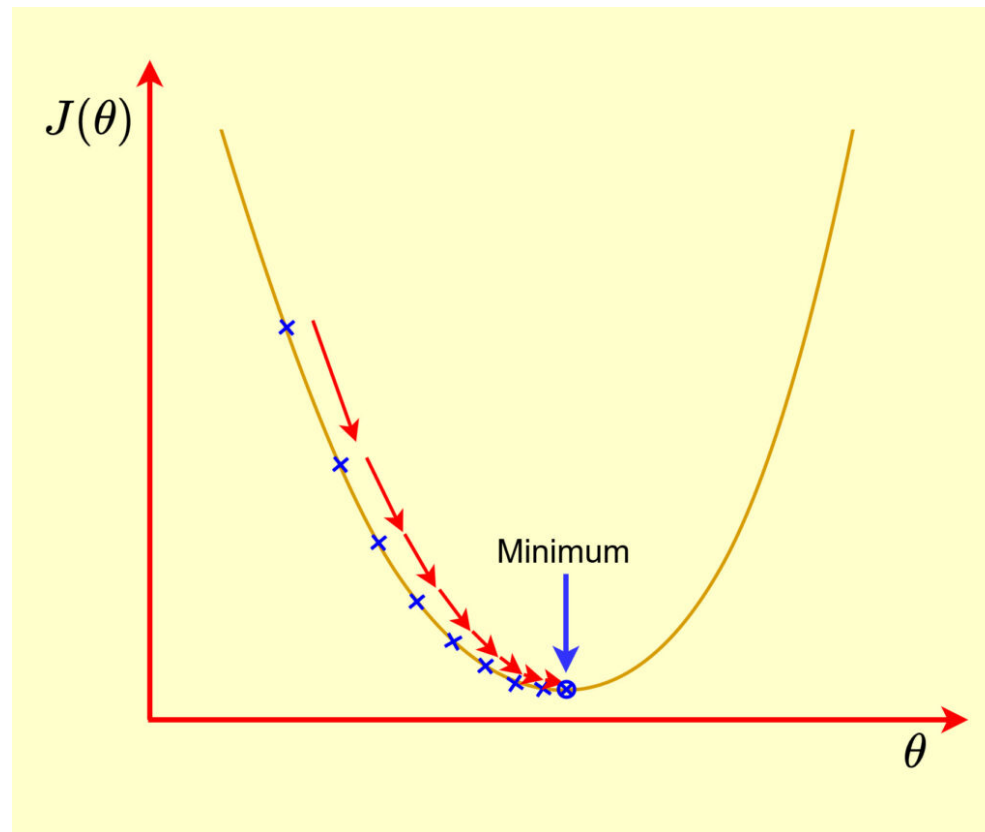
Picking learning rate

- Use grid-search in log-space over small values on a validation set:
 - e.g., 0.01, 0.001, ...
- Sometimes, update after each pass:
 - e.g., decrease by a factor of $1/t$
 - sometimes use cosine annealing
- Fancier techniques we won't talk about:
 - Adaptive gradient: scale gradient differently for each dimension (Adagrad, ADAM,)

Slide courtesy of Matt Gormley

Convexity and Logistic Regression

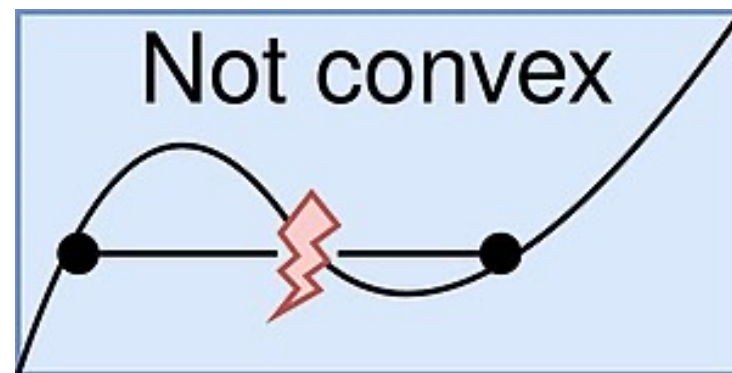
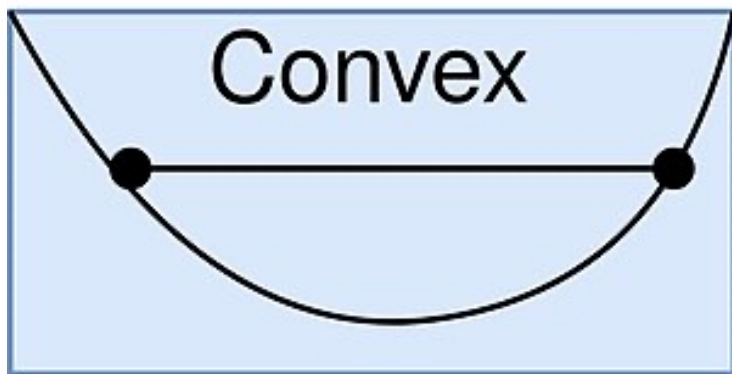
This loss function is convex: there is only one local minimum. So gradient descent will give the global minimum.



Convex function

Definition 1. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if its domain is a convex set and for all x, y in its domain, and all $\lambda \in [0, 1]$, we have

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$



- e^{ax}
- $-\log(x)$

Strict and strong convexity

Definition 2. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is

also known as
Jensen's Inequality

- Strictly convex if $\forall x, y, x \neq y, \forall \lambda \in (0, 1)$

$$f(\lambda x + (1 - \lambda)y) < \lambda f(x) + (1 - \lambda)f(y).$$

- Strongly convex, if $\exists \alpha > 0$ such that $f(x) - \alpha \|x\|^2$ is convex.

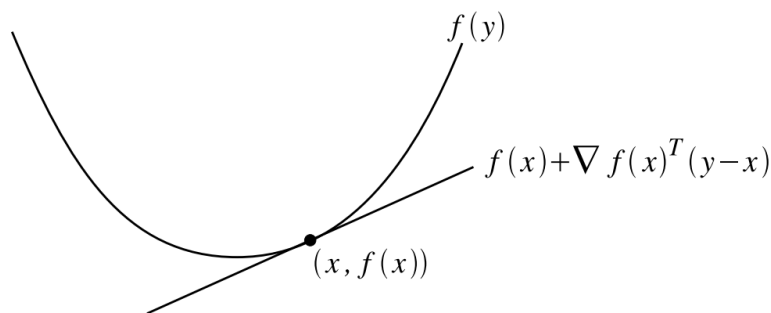
在严格凸的基础上，增加了“弯曲程度”的下界

Lemma 1. *Strong convexity $\Rightarrow$ Strict convexity $\Rightarrow$ Convexity.
(But the converse of neither implication is true.)*

Convex function

Theorem 2. Suppose $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice differentiable over an open domain. Then, the following are equivalent:

- (i) f is convex.
- (ii) $f(y) \geq f(x) + \nabla f(x)^T(y - x)$, for all $x, y \in \text{dom}(f)$.
- (iii) $\nabla^2 f(x) \succeq 0$, for all $x \in \text{dom}(f)$.



Positive semidefinite (半正定)
Hessian matrix

$$\nabla^2 f(x) \succeq 0.$$

First Order Condition for Convexity

Second Order Condition for Convexity

Hessian Matrix

Definition: the **Hessian** of a K-dimensional function is the matrix of partial second derivatives with respect to each pair of dimensions.

$$H_f(\mathbf{x}) := \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_K} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_K \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_K \partial x_2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_K^2} \end{bmatrix}$$

Slide courtesy of Matt Gormley

Hessian Matrix

- Let $f: \mathbb{R}^d \mapsto \mathbb{R}$ be a twice differentiable function. Then, the Hessian of f at $\mathbf{x} \in \mathbb{R}^d$ is a matrix in $\mathbb{R}^{d \times d}$ denoted by $\nabla^2 f(\mathbf{x})$ and defined by

$$\nabla^2 f(\mathbf{x}) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}) \right]_{1 \leq i, j \leq d}$$

- Example: $f(\mathbf{x}) = -\sum_{i=1}^d x_i \ln x_i$

$$\nabla f(\mathbf{x}) = \begin{bmatrix} -(\ln x_1 + 1) \\ \vdots \\ -(\ln x_d + 1) \end{bmatrix} \rightarrow \nabla^2 f(\mathbf{x}) = \text{diag}\left(-\frac{1}{x_1}, \dots, -\frac{1}{x_d}\right)$$

Examples of convex functions

□ Square Loss

- $f(x, v) = (x - v)^2$

□ Absolute Loss

- $f(x, v) = |x - v|$

□ Hinge Loss

- $f(x, v) = \max(0, 1 - xv)$

□ Regularization

- $r(x) = \frac{\lambda}{2} \|x\|_2^2$

- $r(x) = \lambda \|x\|_1$

The Newton's Method

- Gradient descent **may take many steps to converge** to that optimum.
- The motivation behind **Newton's method** is to use a **quadratic approximation** of our function to make a good guess where we should step next.
- From linear regression, we know that we can find the **minimizer** to a **quadratic function** analytically (i.e. **closed form**).

Taylor Series

How can we approximate a function in 1-dimension?

The **Taylor series expansion** for an infinitely differentiable function $f(x)$, $x \in \mathbb{R}$, about a point $v \in \mathbb{R}$ is:

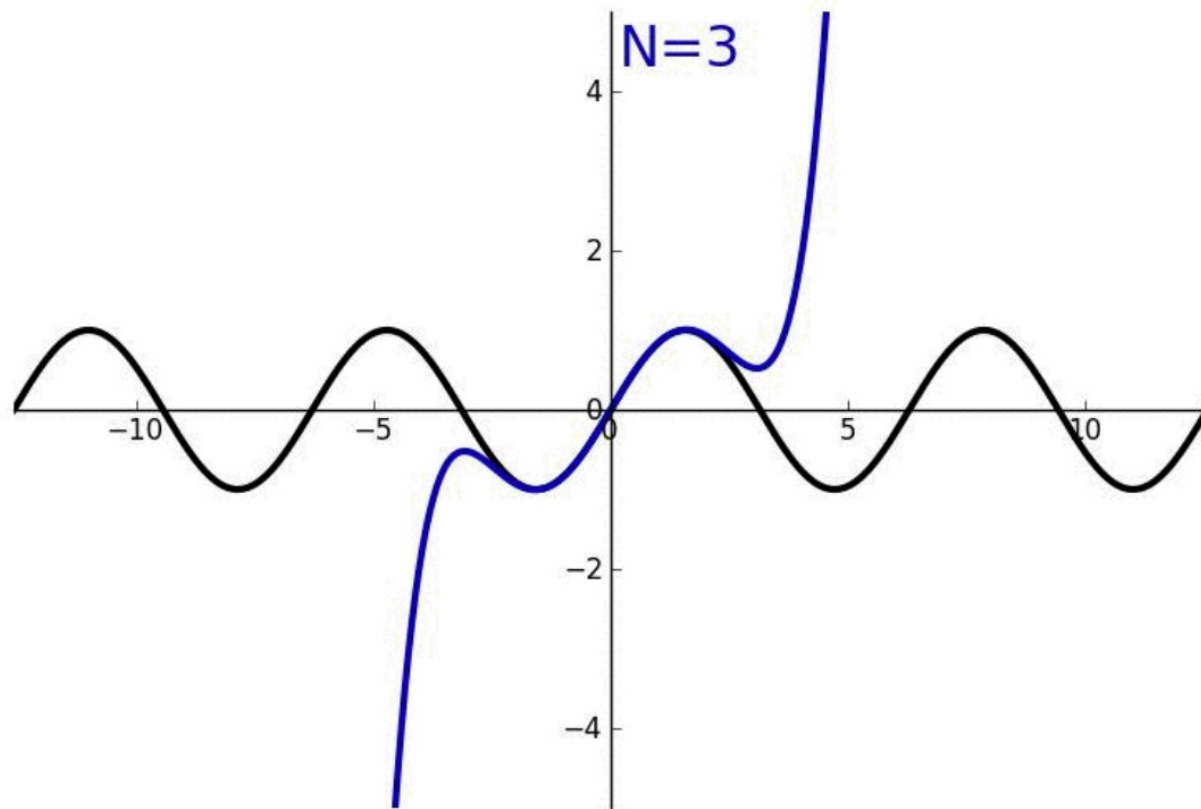
$$f(x) = f(v) + \frac{(x - v)f'(x)}{1!} + \frac{(x - v)^2 f''(x)}{2!} + \frac{(x - v)^3 f'''(x)}{3!} + \dots$$

The **2nd-order Taylor series approximation** cuts off the expansion after the quadratic term:

$$f(x) \approx f(v) + \frac{(x - v)f'(x)}{1!} + \frac{(x - v)^2 f''(x)}{2!}$$

Slide courtesy of Matt Gormley

Taylor Series



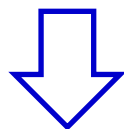
Slide courtesy of Matt Gormley

The Newton's Method

A Taylor expansion around the current point β

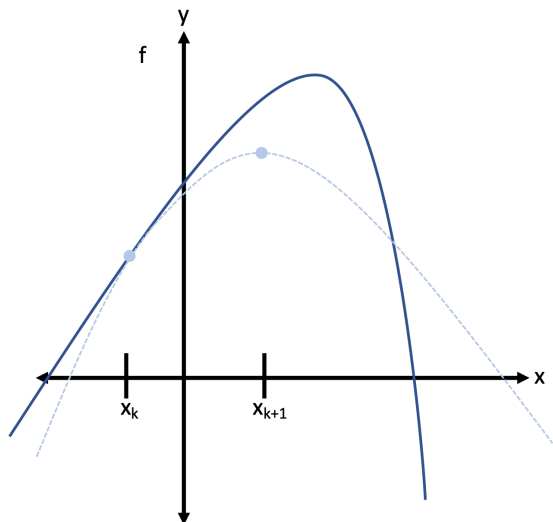
First order: $J(\beta + s) \approx J(\beta) + g(\beta)^\top s$

Second order: $J(\beta + s) \approx J(\beta) + g(\beta)^\top s + \frac{1}{2} s^\top H(\beta) s$



set $\nabla J(\beta + s) = 0$

$$s = -H^{-1}g(\beta)$$



Algorithm 1 Newton-Raphson Method

```
1: procedure NR( $\mathcal{D}, \theta^{(0)}$ )  
2:    $\theta \leftarrow \theta^{(0)}$  ▷ Initialize parameters  
3:   while not converged do  
4:      $g \leftarrow \nabla J(\theta)$  ▷ Compute gradient  
5:      $H \leftarrow \nabla^2 J(\theta)$  ▷ Compute Hessian  
6:      $\theta \leftarrow \theta - H^{-1}g$  ▷ Update parameters  
7:   return  $\theta$ 
```

The Newton's Method

□ We have $\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta}} \ell(\boldsymbol{\beta})$

□ Taking Newton's method as an example, the updating rule at the $(t + 1)$ -th iteration is

$$\boldsymbol{\beta}^{t+1} = \boldsymbol{\beta}^t - \left(\frac{\partial^2 \ell(\boldsymbol{\beta})}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} \right)^{-1} \frac{\partial \ell(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}}$$

where the first- and second-order derivatives with respect to $\boldsymbol{\beta}$ are

$$\frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = - \sum_{i=1}^m \hat{\mathbf{x}}_i (y_i - p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}))$$

$$\frac{\partial^2 J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} = \sum_{i=1}^m \hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) (1 - p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}))$$

Newton's Method for Linear Regression

- Hessian matrix in logistic regression is positive semidefinite

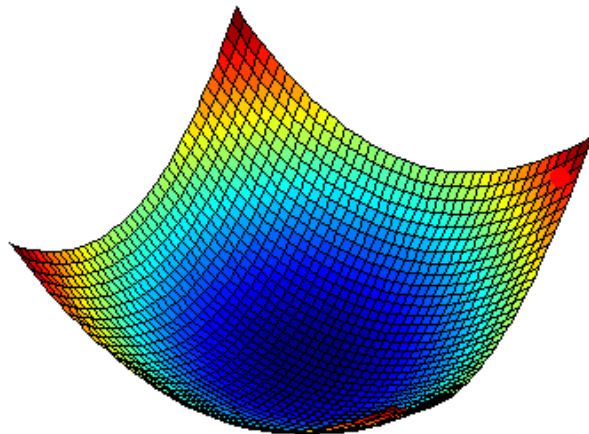
$$\mathbf{z}^T \mathbf{H} \mathbf{z} = \sum_{i=1}^m \underbrace{\|\hat{\mathbf{x}}_i^T \mathbf{z}\|^2}_{\geq 0} \cdot \underbrace{p_1(1-p_1)}_{>0} \geq 0$$

- Newton's method applied to Linear Regression (or any convex quadratic function) **converges in exactly 1-step** to the true optimum.
- This is **equivalent** to solving the Normal Equations
(正规方程)

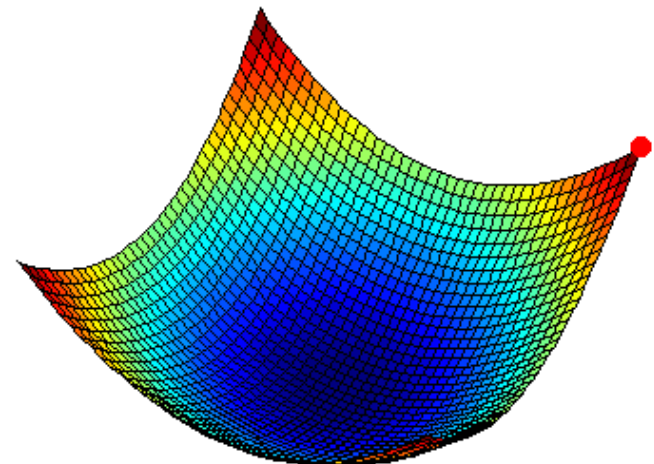
GD vs. Newton's Method

The Newton's method converges much faster often but it's computationally more expensive.

gradient descent iteration 1



Newton iteration 0



Multinomial logistic regression - Softmax

The multinomial logistic classifier uses a generalization of the sigmoid, called the **softmax** function, to compute $p(y_k = 1|\mathbf{x})$.

The **softmax** function takes a vector $\mathbf{z} = [z_1, z_2, \dots, z_K]$ of K arbitrary values and maps them to a probability distribution, with each value in the range $[0,1]$, and all the values summing to 1.

Like the sigmoid, it is an exponential function.

Multinomial logistic regression - Softmax

For a vector $\mathbf{z}$ of dimensionality K , the softmax is defined as:

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^K \exp(\mathbf{z}_j)} \quad 1 \leq i \leq K$$

The softmax of an input vector $\mathbf{z} = [z_1, z_2, \dots, z_K]$ is thus a vector itself:

$$\text{softmax}(\mathbf{z}) = \left[\frac{\exp(\mathbf{z}_1)}{\sum_{i=1}^K \exp(\mathbf{z}_i)}, \frac{\exp(\mathbf{z}_2)}{\sum_{i=1}^K \exp(\mathbf{z}_i)}, \dots, \frac{\exp(\mathbf{z}_K)}{\sum_{i=1}^K \exp(\mathbf{z}_i)} \right]$$

While technically the term is "soft(arg)max", the shortened form "softmax" has become standard terminology in machine learning. This naming reflects its role as a differentiable approximation of the argmax function.

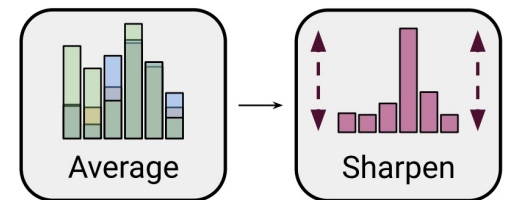
An Example of Softmax

Given a vector:

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

the resulting (rounded) $\text{softmax}(\mathbf{z})$ is

$$[0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$



Like the sigmoid, the softmax has the property of squashing values toward 0 or 1. Thus if one of the inputs is larger than the others, it will tend to push its probability toward 1, and suppress the probabilities of the smaller inputs.

Multinomial logistic regression

- When we apply softmax for logistic regression, we'll need separate weight vectors $\mathbf{w}_k$ and bias b_k for each of the K classes. The probability of each of our output classes $\hat{y}_k$ can thus be computed as:

$$p(y_k = 1 \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}$$

- If we represent the weights in matrix and bias in vectors, we can compute $\hat{\mathbf{y}}$, the vector of output probabilities for each of the K classes, by a single elegant equation:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

Note: for more efficient computation by modern vector processing hardware

Multinomial logistic regression

The cross-entropy loss for a single example $\mathbf{x}$

$$\begin{aligned}\ell_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= - \sum_{k=1}^K y_k \log \hat{y}_k \quad (y_c = 1 \text{ and } y_j = 0, \forall j \neq c) \\ &= - \log \hat{y}_c \quad (\text{where } c \text{ is the correct class}) \\ &= - \log \hat{p}(y_c = 1 \mid \mathbf{x}) \\ &= - \log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}\end{aligned}$$

**negative log
likelihood loss**

Gradient of the weight vector for class k

$$\begin{aligned}\frac{\partial \ell_{\text{CE}}}{\partial \mathbf{w}_k} &= -(y_k - \hat{y}_k) \mathbf{x} \\ &= -(y_k - p(y_k = 1 \mid \mathbf{x})) \mathbf{x} \\ &= - \left(y_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) \mathbf{x}\end{aligned}$$

思考二分类任务中 sigmoid 与 softmax 的关联

$$\begin{aligned}\ell_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= - \sum_{k=1}^K y_k \log \hat{y}_k \quad (y_c = 1 \text{ and } y_j = 0, \forall j \neq c) \\ &= -\log \hat{y}_c \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \hat{p}(y_c = 1 \mid \mathbf{x}) \\ &= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}\end{aligned}$$

线性模型的权重向量是决策边界这个
超平面的法向量

超平面方程: $w^T x + b = 0$

Summary

Data: Inputs are continuous vectors of length d . Outputs are discrete labels.

$$\mathcal{D} = \left\{ \mathbf{x}^{(i)}, y^{(i)} \right\}_{i=1}^m \text{ where } \mathbf{x} \in \mathbb{R}^d \text{ and } y \in \{0, 1\}$$

Model: Logistic function applied to dot product of parameters with input vector.

$$p(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x})} \quad \text{sigmoid function}$$

Learning: finds the parameters that minimize some objective function.

$$\boldsymbol{\beta}^* = \arg \min_{\boldsymbol{\beta}} J(\boldsymbol{\beta})$$

maximum likelihood estimation iterative optimization

Prediction: Output is the most probable class.

$$\hat{y} = \arg \max_{y \in \{0,1\}} p(y \mid \mathbf{x})$$

Questions

- Q1: Modify softmax cross-entropy loss to tackle class imbalanced classification
- Q2: Derive the relationship between cross-entropy and KL divergence
- Q3: Which surrogate function is preferred in multi-label learning, sigmoid or softmax
- Q4: Prove logistic regression with L2 regularizer is a strongly convex problem